



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Diego Alessandro Lopez Higueros **Carné:** 1187525

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es una interfaz de programación que habilita la comunicación entre dos sistemas mediante el protocolo HTTP, bajo los principios de la arquitectura REST. En este modelo, la información se organiza en recursos identificables mediante URLs únicas que se manipulan utilizando los métodos HTTP estándar.

2. ¿Qué es un endpoint?

Es la ruta de acceso específica de una API diseñada para ejecutar una acción sobre un recurso. Surge de la combinación de una dirección URL y un método HTTP determinado.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección web que ubica a un recurso de forma global (por ejemplo, https://miapp.com/api/usuarios/5). El endpoint define la intersección entre esa URL y una acción HTTP específica. Por ende, una única URL puede representar múltiples endpoints según el verbo HTTP que se utilice (como GET o DELETE).

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

- GET: Consulta y recupera información sin alterar el estado del servidor.
- POST: Envía datos para crear un nuevo recurso en el sistema.
- PUT: Sustituye un recurso existente en su totalidad.
- PATCH: Aplica modificaciones parciales sobre un recurso.
- DELETE: Remueve o elimina un recurso determinado.

5. ¿Qué es JSON y por qué es común en las APIs?

JSON (*JavaScript Object Notation*) es un formato liviano de intercambio de datos estructurado en pares clave-valor y listas. Se utiliza masivamente en APIs por ser fácil de interpretar tanto por humanos como por máquinas, consumir poco ancho de banda y ser nativamente compatible con la mayoría de los lenguajes de programación.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es una clave numérica de tres dígitos enviada por el servidor para indicar el resultado de una solicitud HTTP.

- 200 OK: La petición se procesó exitosamente.
- 404 NOT FOUND: El recurso solicitado no existe en el servidor.



- 500 INTERNAL SERVER ERROR: Ocurrió un fallo imprevisto dentro del servidor al procesar la solicitud.

7. ¿Qué información se puede encontrar en la respuesta de una API?

- Código de estado: Notifica si la operación concluyó con éxito o falló.
- Encabezados (*Headers*): Metadatos clave como el formato del contenido (Content-Type), políticas de caché o credenciales de autenticación.
- Cuerpo (*Body*): La carga útil con los datos solicitados (normalmente en JSON) o el detalle del error en caso de fallo.
- Métricas de rendimiento: El tiempo de procesamiento y el tamaño de la respuesta (visibles en clientes como Postman).

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

- Permite validar el funcionamiento de los endpoints sin necesidad de desarrollar la interfaz o el cliente.
- Facilita la definición y ajuste de las estructuras de datos de la API de forma temprana.
- Ayuda a detectar fallos de lógica, rutas o parámetros antes de escribir el código final.
- Permite organizar las peticiones en colecciones reutilizables para compartir con el equipo de trabajo.

9. ¿Qué es y para qué sirve Swagger UI?

Es un entorno visual que convierte especificaciones OpenAPI en documentación web interactiva. Sirve para visualizar de forma clara todos los endpoints, sus parámetros esperados y las respuestas posibles, permitiendo además realizar pruebas en tiempo real desde el navegador mediante la función "Try it out".

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman actúa como una plataforma integral para el diseño, depuración, documentación y automatización de APIs. Su función principal consiste en enviar solicitudes HTTP al servidor, permitir la inspección exhaustiva de las respuestas recibidas y certificar que la API se comporte según lo planificado.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.



- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman



- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las



Universidad
Rafael Landívar

Identidad Jesuita en Guatemala

pruebas realizadas.

Enviar link de github a plataforma de la universidad.

Facultad de Ingeniería

Facultad de Informática y sistemas

Manejo e Implementación de Archivos

Miércoles, 30 de septiembre de 2026